

CHANGELOG

HORIZON GRID

Changelog

Version: 0.1.0

Documentation prepared: 2026-08-17

PREPARED FOR EVALUATION

Table of Contents

HORIZON GRID Changelog	3
Linux Release	6

HORIZON GRID Changelog

All notable changes for the HORIZON GRID release are listed below, grouped by area. Every entry below reflects a real, tested change — not a planned or aspirational one.

Rebrand

- Full visible rebrand from "IOC Intelligence Platform" to **HORIZON GRID** (tagline: "Every Signal. One Operational Picture.") across the frontend, backend API title, Windows installer, Start Menu shortcuts, and all documentation.
- Internal identifiers deliberately left unchanged for upgrade safety: the on-disk ProgramData data folder name, the Postgres database name (`ioc_intel`), Python package names, and the Kubernetes namespace. Only user-visible strings were renamed.

New IOC providers

- Added **urlscan.io** (sandbox category, submit-then-poll scan model) as a new IOC provider.
- Added **Google Safe Browsing** (threat_intel category) as a new IOC provider.
- Both providers were verified, via a real live test with deliberately invalid credentials, to never report a provider failure as a false "safe"/"clean" result — a failed or unreachable call is always surfaced as an error or unknown status.
- A real gap was found and fixed after initial release: both new providers had no live "Test Connection" check wired up (`'<provider>' has no live connection test`) even though the underlying investigation-time provider logic worked correctly. Both now have a real, working connection test.

Export security fixes

- Fixed a CSV formula-injection vulnerability in exported investigation data (a leading `= / + / - / @` in any exported field is now neutralized).
- Fixed a PDF markup-injection/crash vulnerability in exported investigation reports (all interpolated text is now escaped before reaching the PDF renderer).
- Fixed an export permission-gate bug: exporting an investigation now correctly requires the `lookup:export` permission (previously it was gated on the broader `lookup:read`, which VIEWER-role users also hold — VIEWER cannot export).

Deterministic threat-scoring engine

- Replaced the previous 100%-AI-generated risk score with a deterministic, versioned, auditable scoring engine (`app/scoring/engine.py`, `SCORING_ENGINE_VERSION "1.0"`) — see the dedicated Threat Scoring document for the full formula.
- The AI is now given the deterministic score as a fixed input and can only narrate it; the platform mechanically overwrites the AI's own output with the real score and re-validates the full assessment against it before saving, so an AI model cannot override the number even if it tries to.
- A real audit-trail gap was found and fixed: the engine always computed a full factor breakdown and version stamp, but it was being discarded before the assessment was saved. Both are now persisted on every assessment.
- A real scoring-manipulation vulnerability was found via adversarial testing and fixed: a single free, unprivileged account on a community-sourced provider (AlienVault OTX, ThreatFox, or MalwareBazaar) could previously flood the correlation component with

several fabricated, uncorroborated relationship claims and push any indicator's score into the "high" severity band. The correlation component now applies the same cross-provider corroboration discount the provider-vote component already had.

AI-generation outcome tracking

- Every AI-generated assessment is now tagged with one of three real outcomes: `success`, `skipped_no_evidence` (a correct decision not to call the AI — no provider data existed to analyze, not a failure), or `failed` (a genuine generation failure, with a real, deterministic-score-based fallback narrative, never a fabricated result). This distinction did not exist before and makes an honest "AI success rate" metric possible.

Executive Dashboard and Provider Health

- Added a new Executive Dashboard (`/dashboard`) with 7 real KPI tiles (active investigations, critical/high-risk IOC count, open/critical case counts, average threat score, provider health percentage, AI success rate) and an AI-generated (or real, number-accurate template-fallback) executive summary, with the fallback explicitly disclosed via a source badge.
- Replaced the old, dead, stub-data `GET /providers/health` endpoint with a real, database-backed implementation reporting per-provider status (healthy/degraded/down/unknown), success rate, average latency, and consecutive-failure streaks across 1-hour/24-hour/7-day/30-day windows, plus a new dedicated Provider Health page.
- A real bug was found and fixed: a provider correctly reporting "nothing found" for an indicator (the normal, healthy outcome for most real-world lookups) was being miscounted as a failure, which could make a perfectly healthy provider appear "degraded" or "down." Confirmed live: one real provider moved from an incorrectly-reported 64% ("Degraded") to a correctly-reported 100% ("Healthy") once fixed.
- A real regression was found and fixed same-day: the new, richer Provider Health response initially dropped a field (`supported_types`) an existing page depended on, which would have crashed the live investigation-launch page for every user. Fixed and covered by a regression test before it reached general use.
- A new `dashboard:read` permission was added, deliberately granted to all three roles (ADMIN, ANALYST, VIEWER) for broad, read-only operational visibility.

Navigation

- Reorganized the global navigation into named groups: COMMAND (Dashboard), INTELLIGENCE (Basket), ANALYSIS (Cases), OPERATIONS (Provider Health), ADMINISTRATION (Providers, Admin — admin-only). No existing route was changed or removed.

Performance

- Found and fixed a real, complete-failure concurrency bug: 25 concurrent requests to the Provider Health endpoint previously failed 100% of the time (timeout). Root-caused to two issues — roughly 300 sequential database round-trips per single request (consolidated into about 19 via SQL conditional aggregation), and a database connection-pool size that didn't account for every authenticated request holding two connections simultaneously. After both fixes, the same 25-concurrent-request test completes in about 1.6 seconds with 100% success.
- Connection-pool sizing is now tuned per process role: the backend process (the only one serving concurrent dashboard traffic) gets a larger pool; the background worker processes keep a conservative default, since Postgres's own connection ceiling has to be shared across all of them.

Windows installer

- Fixed a real, reported installer failure: a fresh install could generate a brand-new random database password while an old, incompatible database from an abandoned earlier install attempt survived on the machine, causing the backend to crash-loop on a database authentication error immediately after installation. The installer now detects and clears an orphaned database volume before a genuinely fresh install, so a new password is never paired with an old, incompatible database. An upgrade/reconfigure of a real existing install is unaffected and continues to correctly reuse its real existing password.

UI

- Added a small persistent attribution/contact bar at the top of every page.

Linux Release

- Added a first-class Linux release: `horizon-grid_0.1.0_amd64.deb` (~6.1 MB), version 0.1.0 matching Windows exactly, tested on Ubuntu 24.04 LTS, Ubuntu 22.04 LTS, and Debian 12 (bookworm).
- Added a terminal-based CLI setup wizard (`horizon-grid configure`) as the Linux analog of the Windows WinForms wizard, covering Administrator Account, AI Configuration, Threat Intelligence Providers, and Network Ports, then writing `/etc/horizon-grid/.env` and bringing up the platform via Docker Compose.
- Added a `horizon-grid` CLI with `configure` , `start` , `stop` , `restart` , `status` , `open` , `backup` , `diagnostics` , `check` , `uninstall` , and `version` commands.
- Added a systemd unit (`/lib/systemd/system/horizon-grid.service`), registered on install but never auto-started, and a desktop menu launcher (`horizon-grid.desktop` , `Exec=horizon-grid open`).
- Added FHS-standard installed file layout: `/opt/horizon-grid/app` (application), `/etc/horizon-grid/.env` (root:root, chmod 600), `/var/lib/horizon-grid/backups` , `/var/log/horizon-grid/setup.log` .
- Added label-based (not hardcoded-name) cleanup logic for `apt remove` (preserves config/data/volumes) and `apt purge` (removes everything, including all Docker volumes for the Compose project).
- Fixed: excluded two host-only Python virtualenvs (`.venv` , `.venv_test` , ~21,700 files) from being swept into the Linux build by the packaging script.
- Fixed: `apt purge` / `apt remove` leaving behind an untracked runtime-written `/opt/horizon-grid/app/.env` copy that blocked full directory removal; now cleaned up in the package's `postrm` script.
- Fixed: `horizon-grid backup` using a hardcoded Postgres container name, which silently skipped backups under a non-default Compose project name; now resolves the container via `docker compose ps -q postgres` .